Code Seed

LECTURE 02 · FOUNDATIONS

APIs & REST

Designing the contract between our frontend and backend.



Understand the concept. **Then build it.**

THE FOUNDATIONS

Three ideas before we build

DONE

LECTURE 01

How the Web Works

Client · Server · HTTP

YOU ARE HERE

LECTURE 02

APIs & REST

Endpoints · CRUD · JSON

LECTURE 03

Databases

Tables · Relations · SQL · Search

Then → the coding series

THINK ABOUT IT

The frontend wants a list of notes. How does it know what to ask for?

Which URL? Which method? What shape is the answer? Someone has to decide — and write it down.

01

What Is an API

A contract between two programs.

APPLICATION PROGRAMMING INTERFACE

An API is a contract

Interface

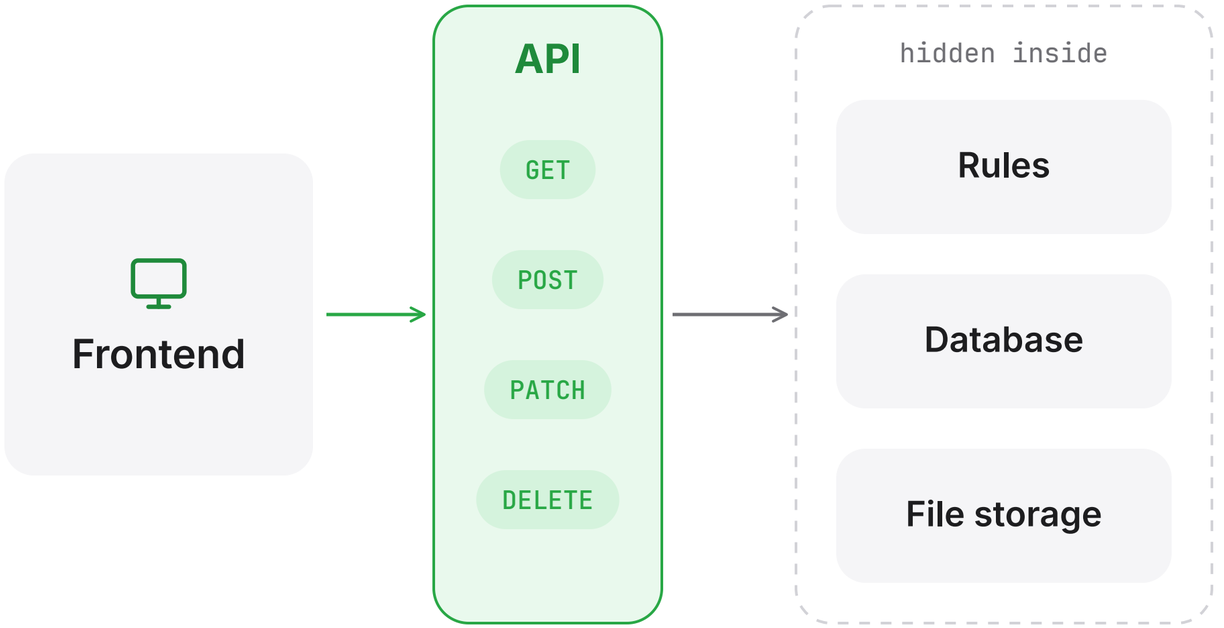
The surface one program exposes to another

Contract

“Send me this, and I promise to return that”

Hides the inside

The frontend never touches the database directly



ANALOGY

Remember the restaurant? The menu is the API

Menu → API

Lists exactly what you can order

Order → Request

You ask using the menu’s words

Dish → Response

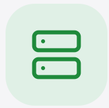
What comes back, as promised

Kitchen → Hidden

How it’s made is not your concern

CONTEXT

APIs are everywhere



Our backend API

What our frontend calls to read and save knowledge

`/api/notes`



Browser APIs

Built into the browser, ready to use from JavaScript

`fetch() · FormData`



Third-party APIs

Other companies’ services you can call

`GitHub · Stripe · Maps`

02

REST

A style for designing web APIs.

REPRESENTATIONAL STATE TRANSFER

REST: a few simple rules

Resources
Every “thing” has a URL: `/notes/42`

Methods are verbs
GET, POST, PATCH, DELETE act on it

Representations
Resources travel as JSON

Stateless
Each request carries what it needs

VERB		NOUN		MEANING
GET	+	/notes	=	List notes
POST	+	/notes	=	Create a note
PATCH	+	/notes/42	=	Edit note 42
DELETE	+	/notes/42	=	Delete note 42

RESOURCES

In our Knowledge Base

The nouns of our Knowledge Base

Notes

Notes, snippets and concepts

`/notes`

Tags

Labels that group ideas

`/tags`

Files

Uploaded PDFs

`/files`

Search

Find across it all

`/search`

Auth

Sign up, log in

`/auth`

NAMING

Nouns in URLs, verbs in methods

✕ AVOID

Verbs in the URL

```
GET /getAllNotes
POST /createNote
POST /deleteNote?id=42
GET /notes/42/update
```

✓ PREFER

Resources + methods

```
GET /notes
POST /notes
DELETE /notes/42
PATCH /notes/42
```

03

CRUD

Four operations behind almost every app.

THE PATTERN

In our Knowledge Base

CRUD: the four basic operations

C

Create

Save a new note

R

Read

Open your notes

U

Update

Edit a snippet

D

Delete

Remove a PDF

MAPPING

One idea, three vocabularies

CRUD	HTTP METHOD	SQL (LECTURE 03)	IN OUR KB
Create	`POST`	`INSERT`	Save a new note
Read	`GET`	`SELECT`	List or open notes
Update	`PATCH` / `PUT`	`UPDATE`	Edit a note
Delete	`DELETE`	`DELETE`	Remove a note

ENDPOINTS

In our Knowledge Base

The Notes API

METHOD	ENDPOINT	WHAT IT DOES	SUCCESS
`GET`	`/api/notes`	List notes	`200 OK`
`GET`	`/api/notes/:id`	Get one note	`200 OK`
`POST`	`/api/notes`	Create a note	`201 Created`
`PATCH`	`/api/notes/:id`	Update some fields	`200 OK`
`DELETE`	`/api/notes/:id`	Delete a note	`204 No Content`

`:id` is a path parameter — a placeholder for a real id, like `42`

CREATE

Create: POST a new note

↗ REQUEST

client → server HTTP

```
POST /api/notes
Content-Type: application/json

{
  "type": "concept",
  "title": "Idempotency",
  "body": "Same request, same result…",
  "tags": ["http", "api"]
}
```

↙ RESPONSE

server → client HTTP

```
HTTP/1.1 201 Created
Location: /api/notes/42

{
  "id": 42,
  "type": "concept",
  "title": "Idempotency",
  "tags": ["http", "api"],
  "createdAt": "2026-10-31T09:30:00Z"
}
```

READ

Read: GET many, or one

↗ MANY

GET /api/notes HTTP

```
HTTP/1.1 200 OK

[
  { "id": 42, "title": "Idempotency" },
  { "id": 41, "title": "Debounce in JS" },
  { "id": 40, "title": "SQL Joins.pdf" }
]
```

↙ MISSING

GET /api/notes/999 HTTP

```
HTTP/1.1 404 Not Found

{
  "error": {
    "code": "NOT_FOUND",
    "message": "Note 999 does not exist"
  }
}
```

UPDATE

PUT replaces. PATCH modifies.

PUT

Send the whole note

missing fields are reset — HTTP

```
PUT /api/notes/42
{
  "type": "concept",
  "title": "Idempotency",
  "body": "Same request, same result…",
  "tags": ["http", "api"],
  "pinned": true
}
```

PATCH

Send only the change

everything else untouched — HTTP

```
PATCH /api/notes/42
{
  "pinned": true
}
```

Our Knowledge Base uses PATCH — editing a title shouldn’t require re-sending the whole note.

DELETE

In our Knowledge Base

Delete: DELETE, then 204

`DELETE /api/notes/42`

No body needed — the URL says it all

`204 No Content`

Success, and nothing to send back

Design choice: soft delete

Move to Trash with a `deletedAt` time, so users can undo

terminal

```
DELETE /api/notes/42
→ 204 No Content

# same request again
DELETE /api/notes/42
→ 404 Not Found
```

IDEMPOTENCY

Safe to repeat?

$$f(f(x)) = f(x)$$

Doing it twice leaves the server in the same state as doing it once.

Method	Safe	Idempotent
GET	✓	✓
PUT	—	✓
DELETE	—	✓
PATCH	—	not always
POST	—	×

THINK ABOUT IT

The Wi-Fi drops while saving. The app retries the POST. What could go wrong?

Hint: POST is not idempotent.

04

Real-World Design

Filtering, files, errors, and auth.

QUERY PARAMETERS

In our Knowledge Base

Query parameters: ask precisely

```
/api/notes?tag=sql&type=snippet&sort=-updatedAt&page=2
```

Collection
Which resource

Filter
Only notes tagged `sql`

Filter
Only code snippets

Sort
`-` means newest first

Paginate
Second page of results

PAGINATION

In our Knowledge Base

Never send everything at once

The problem

10,000 notes in one response = slow app

Page + limit

`?page=2&limit=20` → items 21–40

The formula

`offset = (page - 1) × limit`

Return metadata

So the UI can show “Page 2 of 500”

GET /api/notes?page=2&limit=20 JSON

```json
{
  "data": [
    { "id": 22, "title": "CSS Grid" },
    { "id": 21, "title": "Big-O basics" }
  ],
  "page": 2,
  "limit": 20,
  "total": 10000
}
```

RELATIONSHIPS

In our Knowledge Base

Tags: resources that connect

METHOD	ENDPOINT	WHAT IT DOES
GET	`/api/tags`	List all tags, with note counts
POST	`/api/tags`	Create a tag
GET	`/api/notes?tag=sql`	Notes with a given tag
PUT	`/api/notes/:id/tags`	Replace a note’s set of tags
DELETE	`/api/tags/:id`	Remove a tag from everywhere

FILE UPLOADS

Uploading PDFs

In our Knowledge Base

JSON can’t carry files well

PDFs are binary, not text

`multipart/form-data`

The body is split into parts: the file plus fields

Let the browser set headers

It adds the boundary that separates the parts

upload.js — JAVASCRIPT

```javascript
const form = new FormData();
form.append('file', input.files[0]);   // the PDF
form.append('title', 'PostgreSQL Indexing');

const res = await fetch('/api/files', {
  method: 'POST',
  body: form,   // no Content-Type header here!
});
// → 201 { id: 7, name: "indexing.pdf" }
```

UPLOAD PIPELINE

In our Knowledge Base

What happens to an uploaded PDF

1

Receive

`POST /api/files` arrives with the file

2

Validate

Is it a PDF? Under 20 MB? Logged in?

3

Store bytes

Save the file to disk or cloud storage

4

Save details

Name, size, owner, path → database

5

Extract text

Pull out the words so search can find them

One request, five jobs. Steps 3–5 are explained in Lecture 03.

SEARCH

In our Knowledge Base

One search box, one endpoint

`GET /api/search?q=react+hooks`

Notes, snippets, concepts and PDF text

Ranked results

Best matches first, with a preview

How it works

Full-text search in the database — Lecture 03

response JSON

```
{
  "query": "react hooks",
  "results": [
    { "type": "concept", "id": 42,
      "title": "React Hooks explained",
      "preview": "…hooks let components…" },
    { "type": "pdf", "id": 7,
      "title": "React Docs.pdf",
      "preview": "…rules of hooks…" }
  ]
}
```

ERRORS

Errors: consistent and helpful

Validate everything

Required fields, types, lengths, allowed values

Use the right status

`400` bad input · `401` not logged in · `404` missing

One error shape

The frontend handles every error the same way

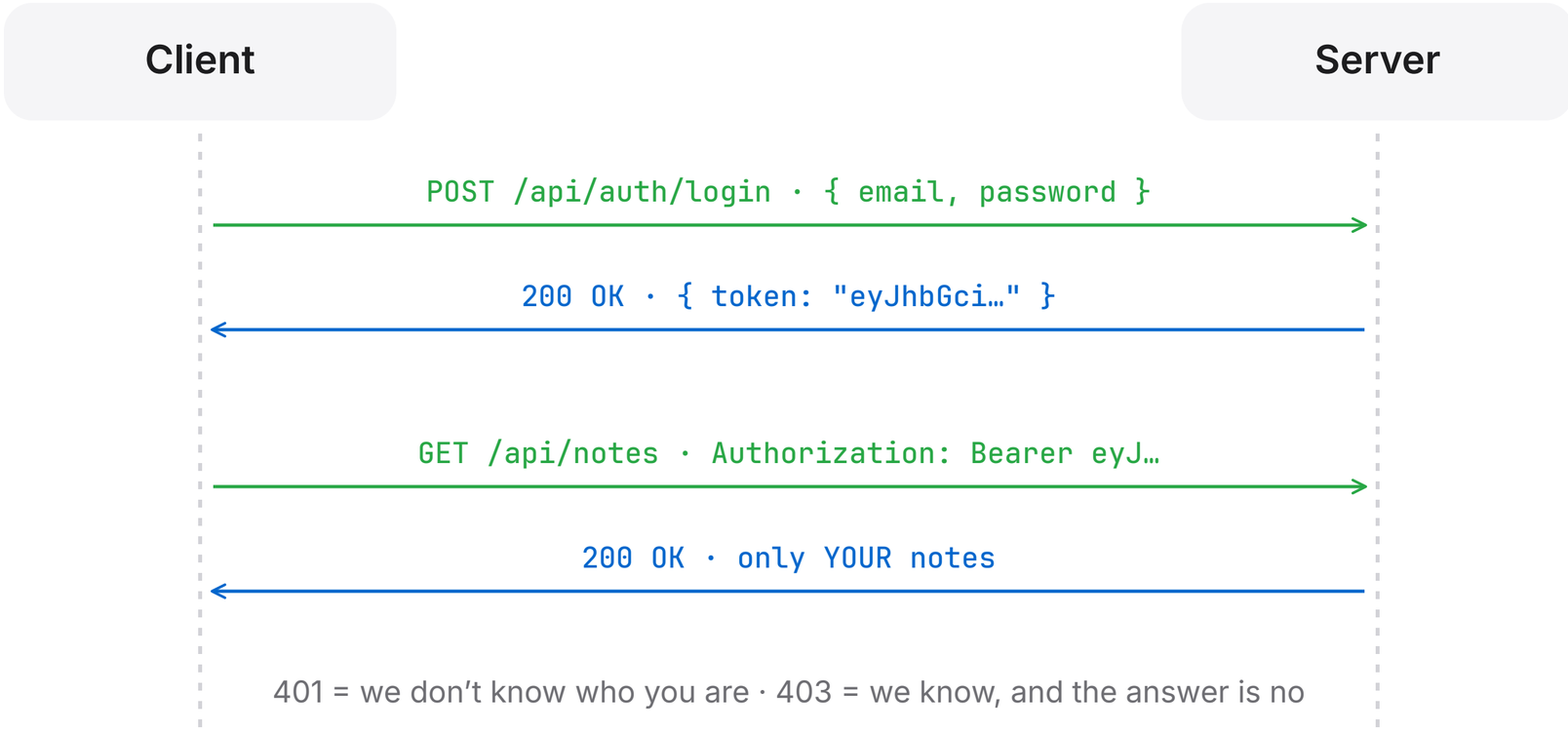
400 Bad Request — JSON

```json
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Some fields are invalid",
    "fields": {
      "title": "Title is required",
      "type": "Must be note, snippet or concept"
    }
  }
}
```

AUTHENTICATION

In our Knowledge Base

Who are you? Tokens



IMPLEMENTATION

From contract to code

routes/notes.js JAVASCRIPT

```
import { Router } from 'express';
const notes = Router();

notes.get('/',       listNotes);    // GET    /api/notes
notes.get('/:id',    getNote);      // GET    /api/notes/42
notes.post('/',      createNote);   // POST   /api/notes
notes.patch('/:id',  updateNote);   // PATCH  /api/notes/42
notes.delete('/:id', deleteNote);   // DELETE /api/notes/42

app.use('/api/notes', requireAuth, notes);
```

Router

Groups related endpoints

`:id`

Arrives as `req.params.id`

Middleware

`requireAuth` runs before every route

TESTING

Test the API before any UI

terminal TERMINAL

```
# Create a note
curl -X POST http://localhost:3000/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "REST", "body": "Resources + verbs"}'

# List notes with a tag
curl "http://localhost:3000/api/notes?tag=api"

# Delete it
curl -X DELETE http://localhost:3000/api/notes/42
```

Prove it works alone

Backend first, then the UI

Tools

curl · Postman · Bruno · VS Code REST Client

Read the status

It tells you who to blame

BLUEPRINT

In our Knowledge Base

Our complete API — on one slide

Notes

GET	/notes
GET	/notes/:id
POST	/notes
PATCH	/notes/:id
DELETE	/notes/:id

Tags

GET	/tags
POST	/tags
PUT	/notes/:id/tags
DELETE	/tags/:id

Files

GET	/files
POST	/files
GET	/files/:id
DELETE	/files/:id

Search & Auth

GET	/search?q=
POST	/auth/signup
POST	/auth/login
GET	/auth/me

RECAP

Lecture 02 in six ideas

01 **An API is a contract**
What you can ask, and what you get back

02 **REST = nouns + verbs**
URLs name resources, methods act on them

03 **CRUD everywhere**
Create · Read · Update · Delete

04 **Status codes are part of the API**
201, 204, 400, 401, 404 — use them precisely

05 **Repeat safely**
Know which methods are idempotent

06 **Design for reality**
Filters, pagination, uploads, errors, auth

NEXT · LECTURE 03

Databases

Our API promises to remember notes, tags and PDFs. Next: where that data lives, how it connects, and how we search it.

Homework: design the endpoints for a “Bookmarks” feature